TEAM LGTM

FilsPay Handover

Audit, fix and finish a Bahrain card-payments platform, with an AI harness

109
findings confirmed

88
fixed, test-first

22
refund gaps closed

Team LGTM

AN — Akheel Nazim

Owned

- Auditing
- Planted traps
- AI workflow design
- Fixes

AK — Ahmed Khalaf

Owned

- Fixes
- Commit history audit
- Deployment
- Load testing
- UI fixes

Plus Claude Code: Opus for fixes, Haiku for failing tests, Sonnet as reviewer

Root cause: the rules lived on paper, not in the code

Integer fils, unique payouts and tenant checks were written in ADRs and postmortems, but nothing in the code or the database enforced them.

1

Money maths drifted

TS and Go rewrote fils logic with floats, cents and int32

2

"Fixed" races weren't

Postmortems listed unique indexes and locks that never existed

3

Checks on the route, not the query

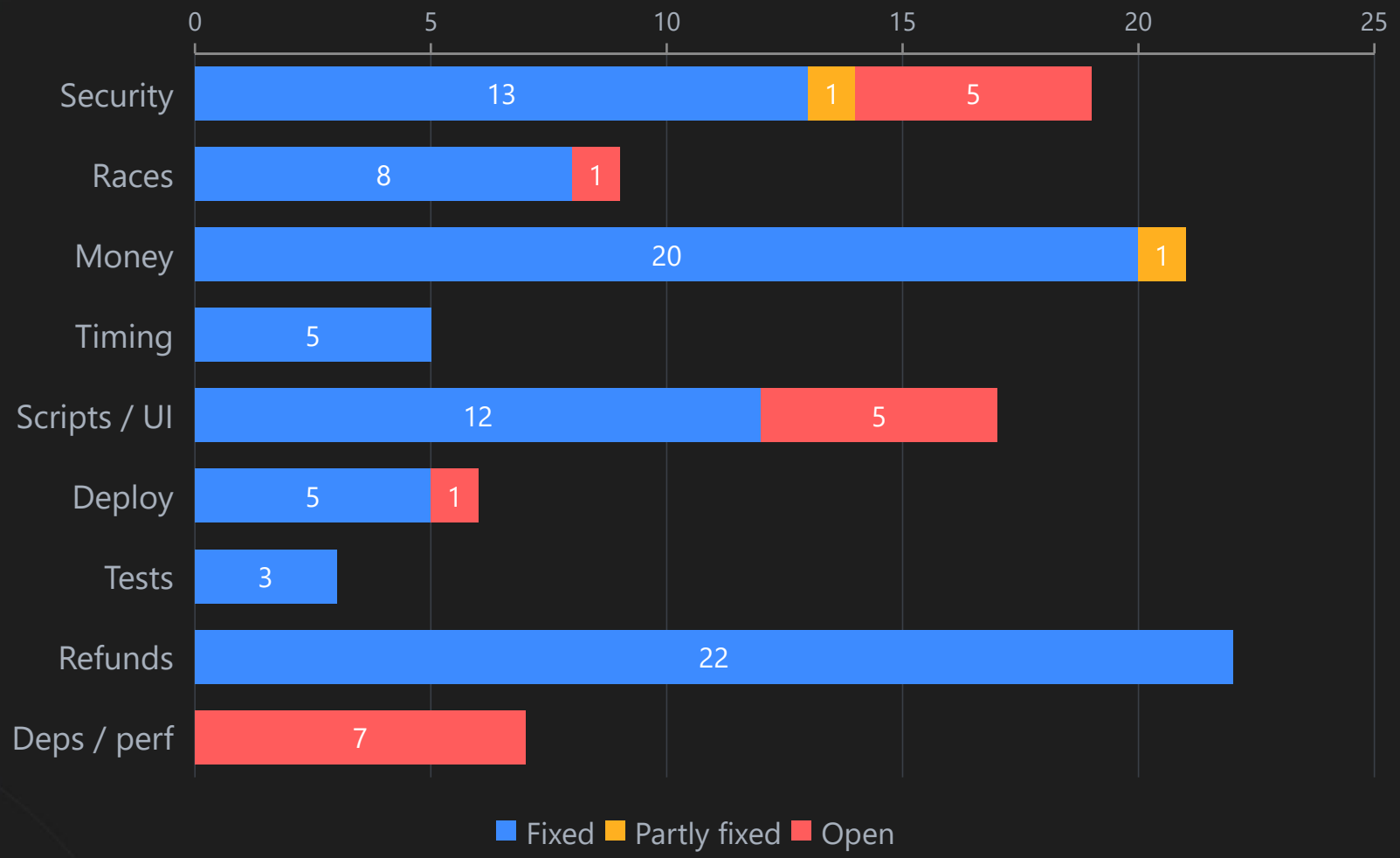
Any id from another merchant worked

4

Even the AI rules were planted

CLAUDE.md said cents, UTC, "don't fix refunds"

Where the bugs were: 109 findings by class



88

fixed

2

partly fixed

19

open, all recorded

Key findings, ranked by impact

1	**Money left twice**	Two refunds of 6,000 on a 10,000 capture both succeeded; a payment link charged two buyers	F3 · R1
2	**Login skipped the OTP**	The pre-OTP cookie worked on every route, reproduced on the running API	S1 · S2
3	**Fils lost or invented**	CSV import booked 4.75 BD as 0.475 BD; volume counter wrapped and switched off the fee tier	M1 · M6
4	**Refunds never worked**	Route answered 501 and every insert failed: 22 gaps behind "just wire it"	F1–F22

Fixes and proof: a failing test before every fix

Confirm in code → Failing test → Smallest fix → Reviewer agent → One commit

Fix	Before	After
S1 pending session	Reaches /auth/me	**Refused**
S4 expired link, old submittedAt	200, charged	**410**
M1 CSV "4.75" BD	475 fils	**4,750 fils**
F8 capture 10,000, refund 4,000	Paid 9,750	**Paid 5,850**
Test suite baseline	16 failing	**0 of those failing**

93 fix test files

12 new migrations

1 commit per bug

Every entry: symptom, cause, fix, test, data effect in fix-docs/FIXLOG.md

Live demo: a partial and a full refund, end to end

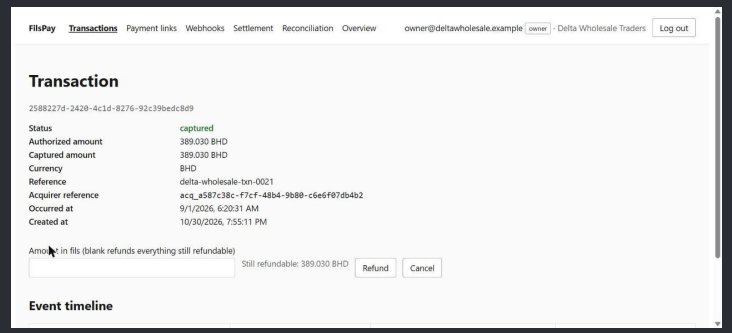
Captured → 389.030 BHD

Partial refund → 100.000 BHD

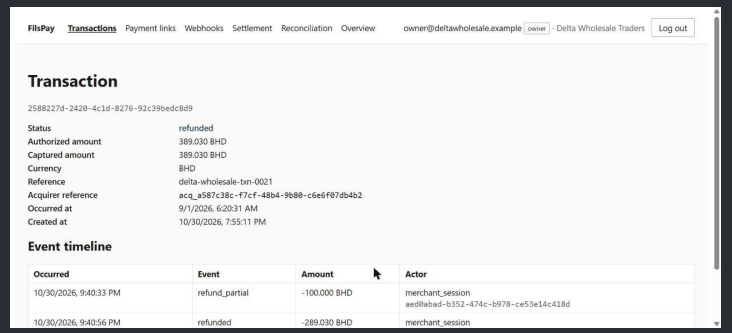
Full refund → remaining 289.030 BHD

Dashboard → status refunded

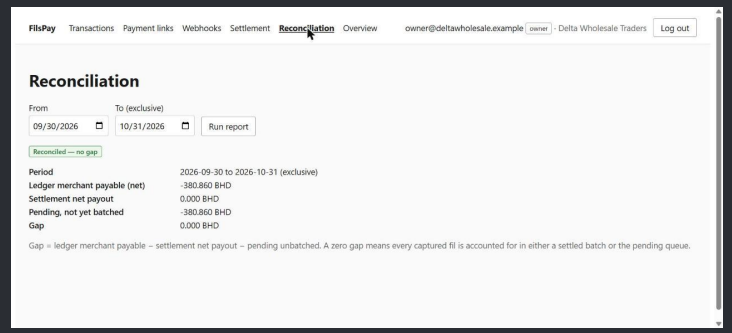
Reconciliation → no gap



Drawer caps the refund at what's left: 389.030 BHD



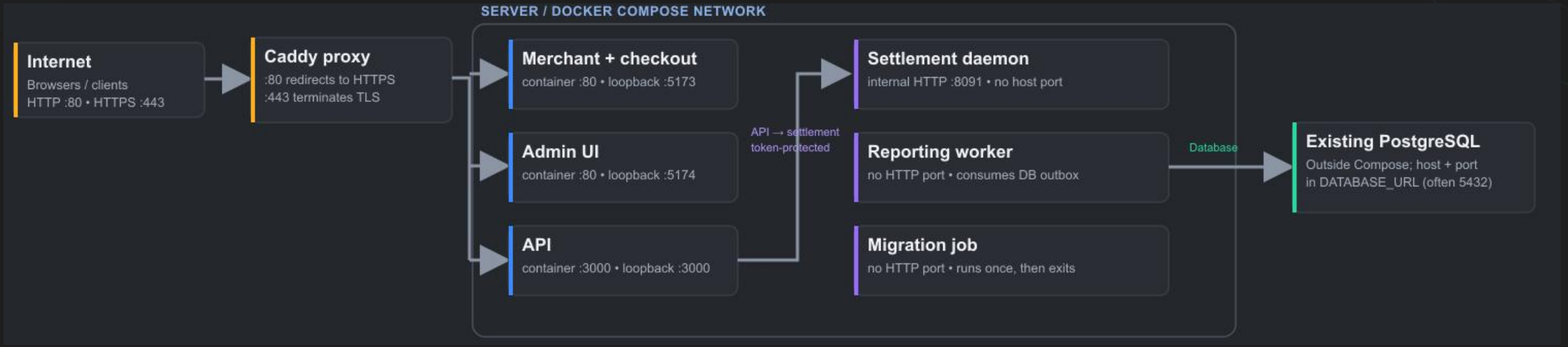
100.000 then 289.030 BHD back: status refunded



Reconciliation after the refunds: no gap

Recorded live on merchant.139-59-92-10.sslip.io (transaction delta-wholesale-txn-0021) · full recording: LGTM_FilsPay_live_refund_flow.gif

Under load



Simulated users

28 estimated (FP-209)

200 measured, 0 errors

At 200 users (k6, live API)

123.4 req/s

1.38 s p95 latency

0 of 19,610 requests failed

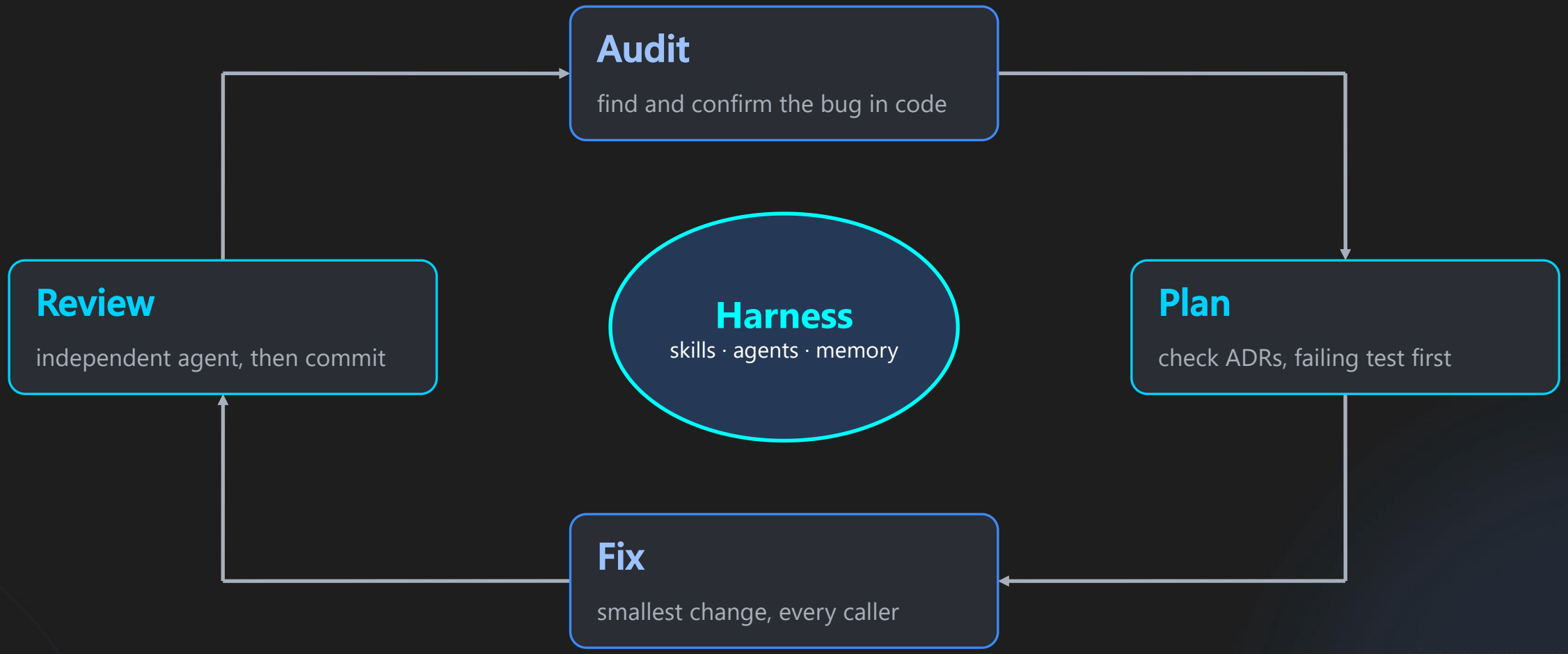
Bottleneck: the database; requests wait on unindexed scans (P2)

Overload: no rate limits yet, so load queues instead of failing

Recovery: back to ~204 ms at 10 users, no restarts

How we used AI: one cycle, every bug

Claude Code ran inside a harness we built: skills, subagents, rules and memory that every session loads.



Inside the harness

Floorplan

Interactive map of every module, file and endpoint, used to understand the code

Clean rules

Deleted the planted CLAUDE.md, AGENTS.md and .cursorrules; ADRs are binding

fix-and-build skill

Test-first, smallest fix, check TS and Go callers, one commit per bug

Right model per job

Haiku writes failing tests, Opus fixes, Sonnet reviews every fix

Shared memory in the repo

fix-docs/memory: commit rules, decisions, pitfalls, so every session and teammate starts the same

Git autonomy

Branch per fix, pull, squash-merge, push; conflicts with a teammate recorded

When the AI was wrong

Opus's first fix for split payouts (M3) could pay 4 fils on a 3-fil payout. The Sonnet reviewer built the five-way case that broke it; we switched to largest-remainder allocation and added the case as a test.

Decisions and open risks

Chose not to change or run

- Kept ADR-0004: rejected the audit's M9 shortcut and fixed the settled-data path
- Never ran scripts/ one-offs against data
- Didn't overwrite payout estimates or past ledger entries without Finance (M19, M14, X14)
- Smallest fix first; the bigger fix is recorded per bug

Still open

- S14 SSRF and S15 dev-mode compose: fix before real traffic
- Rotate the leaked acquirer secret (S11)
- March sweep production postings (X14)
- 16 more recorded items, mostly dependencies and performance
- Demo shows OTP codes on the login screens: demo data only

Thank you

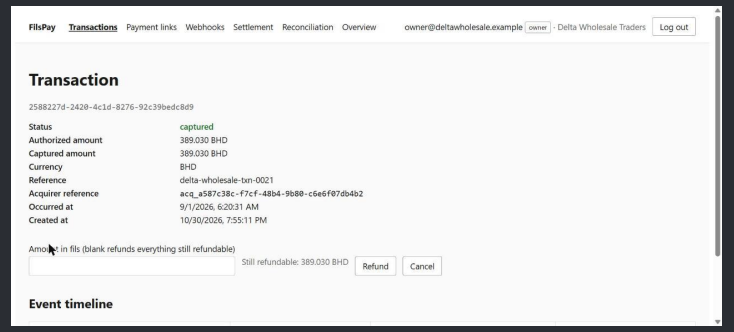
Team **LGTM** · Akheel Nazim · Ahmed Khalaf

Merchant + checkout	**merchant.139-59-92-10.sslip.io**
Admin console	**admin.139-59-92-10.sslip.io**
API	**api.139-59-92-10.sslip.io**
Code	**github.com/akheelnazim/fils-pay**

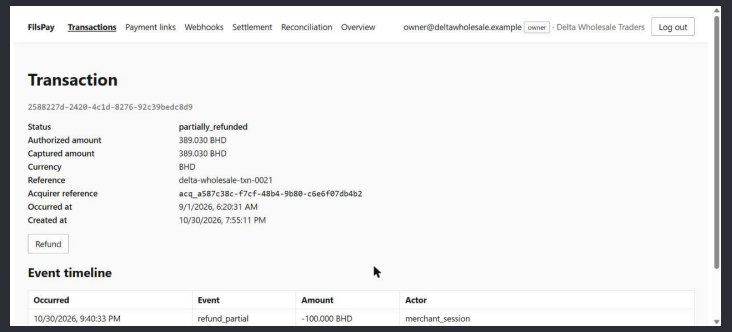
Appendix

Not presented. Full detail in LGTM_FilsPay_Submission.pdf and fix-docs/FIXLOG.md

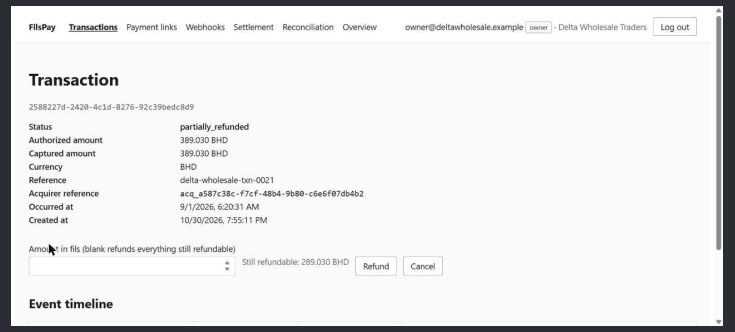
Live demo backup: every step, on the deployed site



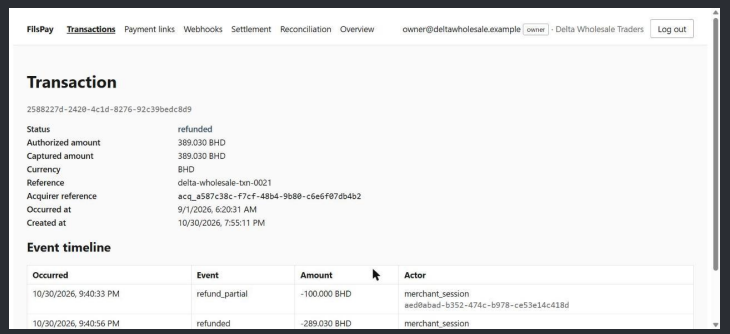
1. Refund drawer: 389.030 BHD refundable



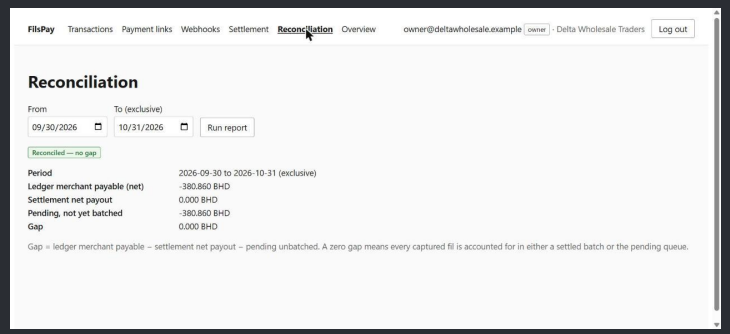
2. Refund 100.000 BHD: partially_refunded



3. Drawer now caps it at 289.030 BHD



4. Blank amount refunds the rest: refunded



5. Reconciliation: gap 0.000 BHD

Proof: tests, migrations, end-to-end refund

Baseline: 16 tests failing before any fix

- 11 refund tests: every insert failed (F14)
- 1 webhook backoff test: exact assert on random jitter (Q1)
- 4 reporting tests: polluted shared DB, consumer stalled (Q2, X6)
- End-to-end refund test: ledger balances, full fee reversed, signed webhooks, reconciliation gap 0

Now: 669 / 669 TypeScript tests pass
API 578 · money 40 · reporting 18 · web 12 · scripts 16 · Go 8/8 packages

12 new migrations (0073–0084)

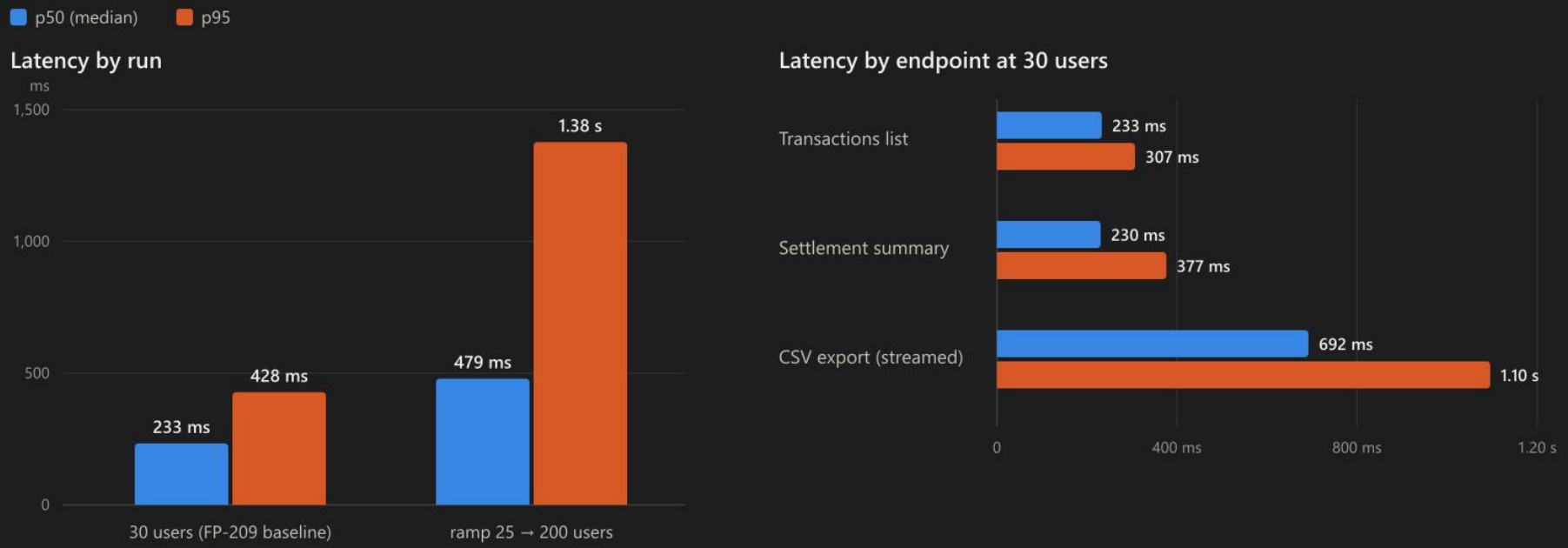
BIGINT money columns (M5, M6) · unique indexes against duplicates (R2, R3, R5) · reporting dead letters (X6) · ADR-0005 renumbering (D1) · integer scheme export (M16) · capture marker (R6) · refund columns, view and pending rows (F9, F15, F22)

Load test results

Load test on the live deployment

k6, 30 Oct, against api.139-59-92-10.sslip.io · Delta Wholesale data · 19,610 requests, 0 failed

200	123.4	1.38 s	0 / 19,610
peak simulated users	requests/s on the ramp to 200	p95 latency at the 200-user ramp	requests failed, both runs
estimate was 28 · 0 errors	60.0 req/s at 30 users	428 ms at 30 users	every status check passed



Requests per endpoint, both runs: Transactions list 13,503 · Settlement summary 3,493 · Checkout link lookup 1,762 · CSV export (streamed, X3) 848. Source: loadtest/summary-fp209.json, summary-ramp.json